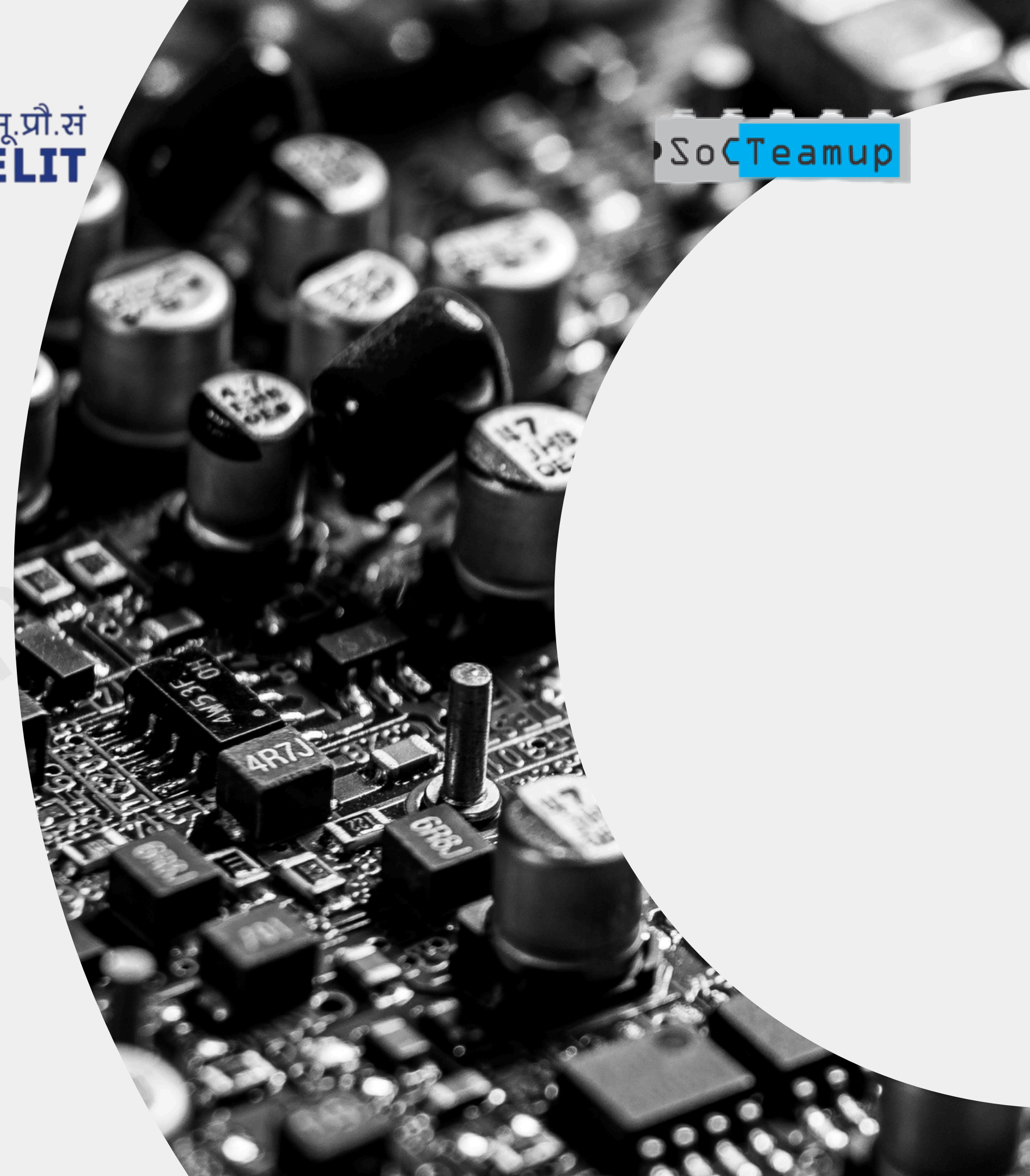


CDC(Clock Domain Crossing) and RDC(Reset Domain Crossing) handling





Outline

Introduction to CDC and RDC
What is Metastability? Why is It Dangerous?
When Do Clock Domain Crossings Happen?
CDC Synchronization Techniques – Control Signals
CDC Synchronization Techniques – Multi-bit Data
Asynchronous FIFO for CDC – Internal Architecture
RDC – Reset Domain Crossing Explained
Reset Synchronizer and Pulse Sync Techniques
CDC/RDC Simulation and Formal Verification
Summary – CDC/RDC Design Best Practices

Introduction to CDC and RDC

- **CDC: Clock Domain Crossing**
 - When Signal transfer between two asynchronous clock domains
- **RDC: Reset Domain Crossing**
 - Reset signal used in domain different from its origin
- **Impacts**
 - Metastability
 - Glitches
 - Lost or corrupted signals
 - Hard-to-reproduce bugs in silicon

➔ What is **Metastability**? Why is It **Dangerous**?

- Occurs when flip-flop samples input during transition (setup/hold violation)
- Output may oscillate or remain unstable
- Not always visible in simulation

Results in:

- Random behavior
- Intermittent failures
- Unreliable operation

When Do **Clock Domain** Crossings Happen?

- **CDC occurs when:**
 - Signals go from clk_A to clk_B
 - IPs run on different clocks
 - Crossing between fast/slow or async blocks
- **Examples:**
 - CPU (clk_core) to UART (clk_uart)
 - AXI master to APB slave
 - External inputs (buttons, sensors)

CDC Synchronization – Control Signals

- **Safe Methods:**
 - 2FF Synchronizer – best for 1-bit pulse or level
- **Design Tips:**
 - Place synchronizer in destination domain
 - Use registered inputs
- **Structure:**
 - flip-flop → use synchronized signal

CDC Synchronization – Multi-bit Data

Common Techniques to Safely Transfer Multi-bit Data :

- **Handshake Mechanism :**
 - Valid and ack signals between source and destination
- **Toggle or Gray Code Encoding:**
 - Convert binary counter to Gray code to avoid multi-bit toggles (asynchronous FIFOs).
- **Strobe-Based Sampling:**
 - Indicates when new data is ready, destination domain synchronizes and captures

Asynchronous **FIFO** for CDC – Internal Architecture

Key components

Dual-port memory -
Shared memory
accessible by both
domains

Write and read
pointers - Tracks
write/ read address

Pointer
synchronizers-Safel
y transfer
read/write pointers
across domains

Asynchronous **FIFO** for CDC – Internal Architecture

Benefits

Reliable data transfer

Handles backpressure (full/empty flags)

Used in DMA, video, audio, AXI-streams

RDC – Reset Domain Crossing Explained

- **Why dangerous?**
 - Reset affects sequential logic
 - Crossing reset between domains → glitches
 - Improper release → unpredictable state
- **Common Cases:**
 - Debug reset from CPU to peripheral
 - Global reset affecting async IPs

Reset Synchronizer **and** Pulse Sync Techniques

- **Best Practices:**
 - Assert reset asynchronously (immediate)
 - De-assert synchronously (safe reentry)
- **Techniques:**
 - 2FF Reset Synchronizer
 - Pulse Sync Circuit (for 1-cycle reset pulse)
 - MUX-based glitch-free reset logic
- **Tools:**
 - Lint + Formal tools + Gate sim

Summary – CDC & RDC Best Practices

- Identify domain crossings during design phase
- Use 2FF for control, Async FIFO for data
- Never mix clocks directly without synchronization
- Sync reset de-assertion in destination domain
- Use CDC/RDC static checkers (Spyglass, Questa CDC, Symbi Yosys)

Note: there is no open source tool to handle end to end CDC issues

Tip: Document every CDC & RDC path in SoC spec

Thank You

echiphu

